PROJECT REPORT

Credit Score Classification

Predicting Good, Standard and Poor credit scores with a Decision Tree Classifier

Project ID	P35
Project Title	Credit Score Class (Decision Tree Classifier)
Name	Bhavik Anjani
Roll No.	B-01
Enrollment No.	24004501210016
Division	D

1. Problem Statement

Banks and credit bureaus must quickly decide whether a customer is a good, average or risky borrower. Manual assessment is slow and inconsistent, so a data-driven classifier can support faster and fairer decisions. This project (**P35 - Credit Score Class**) builds a supervised, multi-class classification model that predicts a customer's **Credit_Score** category (**Good**, **Standard** or **Poor**) from their financial profile and payment behaviour: income, debts, loans, payment delays, credit mix and related fields.

A **Decision Tree Classifier** (scikit-learn) was chosen because it is interpretable: it shows which factors drive each prediction. Models are compared on a 20% hold-out test set using accuracy, macro precision, macro recall, macro F1, specificity and macro one-vs-rest AUC. Because the classes are imbalanced, **macro F1** is the metric used to pick the final setting.

2. Dataset Description and Data Dictionary

The data comes from the downloaded file **train.csv**: **3,380 records and 28 columns**, covering **423 customers observed monthly from January to August**. It mixes numeric, categorical and text fields and is deliberately messy (placeholder strings such as "_" and "!@9#%8", impossible ages such as -500, and missing values in 12 columns). The target is **Credit_Score**: Standard 1,784 (52.8%), Poor 948 (28.1%), Good 647 (19.1%), with 1 record missing the label.

Column	Type	Description	Missing
ID	ID	Unique record ID (dropped)	0
Customer_ID	ID	Customer identifier, 423 unique (dropped)	0
Month	Categorical	Month of the record (January to August)	0
Name	Text	Customer name (dropped)	329
Age	Numeric	Customer age in years	0
SSN	ID	Social security number (dropped)	0
Occupation	Categorical	Customer's occupation (16 categories)	0
Annual_Income	Numeric	Yearly income	0
Monthly_Inhand_Salary	Numeric	Monthly take-home salary	528
Num_Bank_Accounts	Numeric	Number of bank accounts held	0
Num_Credit_Card	Numeric	Number of credit cards held	0
Interest_Rate	Numeric	Interest rate on credit/loans (%)	0
Num_of_Loan	Numeric	Number of loans taken	0
Type_of_Loan	Categorical	Types of loans taken (comma separated)	440
Delay_from_due_date	Numeric	Average days of delay from payment due date	0
Num_of_Delayed_Payment	Numeric	Number of payments delayed	223
Changed_Credit_Limit	Numeric	Percentage change in credit limit	0
Num_Credit_Inquiries	Numeric	Number of credit inquiries	74
Credit_Mix	Categorical	Quality of the credit mix (Good / Standard / Bad)	0
Outstanding_Debt	Numeric	Remaining debt owed	0
Credit_Utilization_Ratio	Numeric	Share of available credit being used (%)	0
Credit_History_Age	Text	Length of credit history, e.g. 22 Years and 1 Months	308
Payment_of_Min_Amount	Categorical	Whether only the minimum amount is paid (Yes / No / NM)	1
Total_EMI_per_month	Numeric	Total monthly EMI (instalment) payments	1
Amount_invested_monthly	Numeric	Amount invested each month	141
Payment_Behaviour	Categorical	Spending / payment pattern (7 categories)	1
Monthly_Balance	Numeric	Month-end balance	47
Credit_Score	TARGET	Credit score class: Good / Standard / Poor	1

3. Data Cleaning and Exploratory Data Analysis

Cleaning steps (notebook Step 5 and Step 7):

Issue found	Action taken
0 duplicate rows	Checked; nothing to remove.
Identity columns: ID, Customer_ID, Name, SSN	Dropped (no predictive value, privacy risk). 28 columns became 24.
Junk placeholders ("_", "_____", "!@9#%8") and stray underscores	Stripped and converted to missing values.
Impossible ages (e.g. -500; outside 18-100)	Set to missing, then imputed.

Issue found	Action taken
Missing values in 12 columns	Median for numeric columns, mode for text columns. 0 missing values remain.
1 row with no Credit_Score	Dropped. Final dataset: 3,379 rows.
Feature engineering	New feature **Debt_to_Income_Ratio** = Outstanding_Debt / Annual_Income (zero income handled, median fill).
Encoding, split and selection	Target label-encoded; categoricals one-hot encoded; stratified 80/20 split (2,703 train / 676 test, random_state=42); **SelectKBest (f_classif, k=15)** fitted on the training set only.

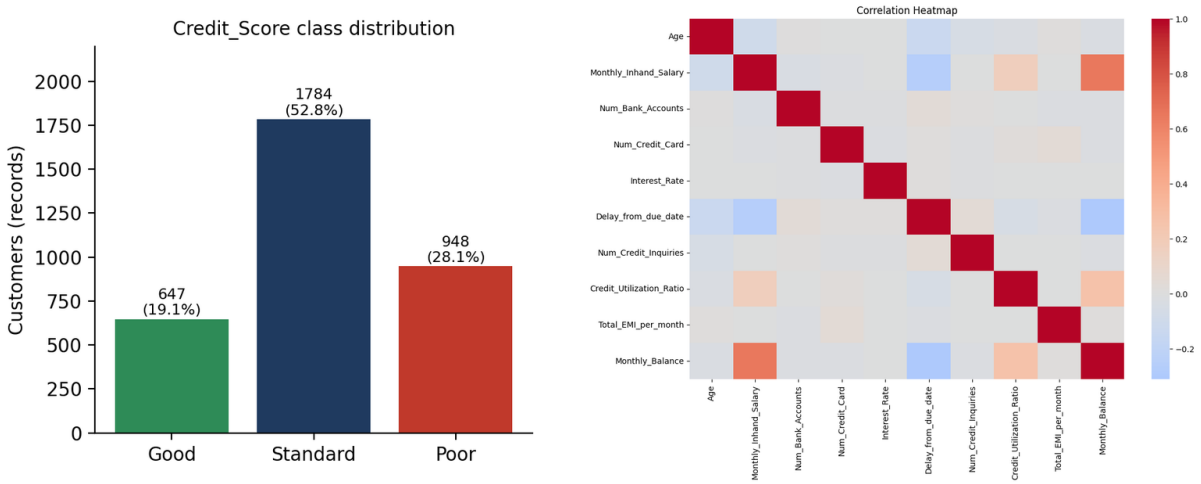


Figure 1. Class distribution (left) and correlation heatmap of numeric features (right).

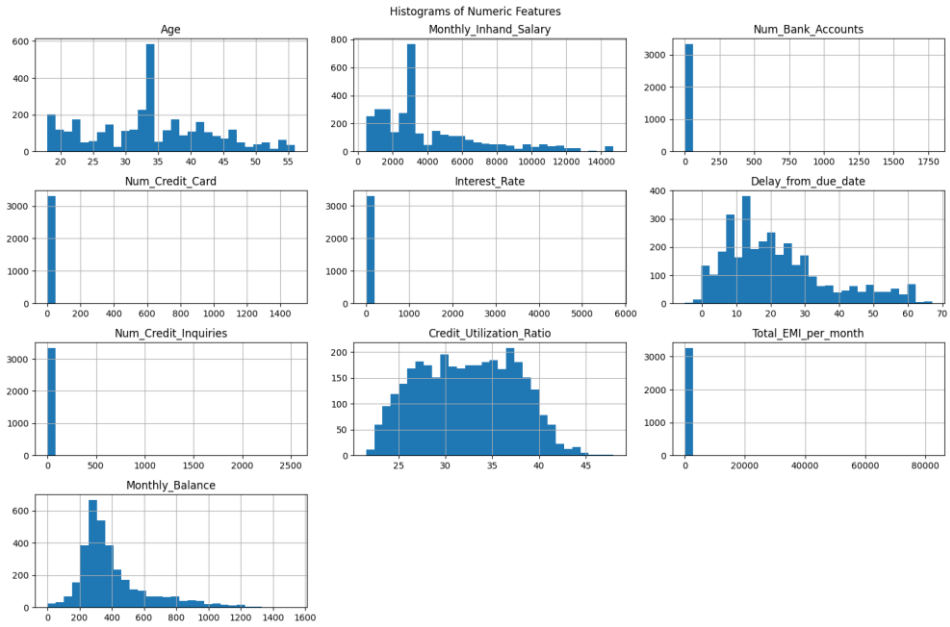


Figure 2. Histograms of numeric features after cleaning.

Key insights from the charts

- **Class imbalance:** Standard is 52.8% of records, so a model that always says "Standard" would already score about 53% accuracy. This is why macro F1, not accuracy alone, is used.
- **Imputation artefacts:** the spike at age 33-34 and the spike near a salary of 3,000 are caused by median imputation of invalid and missing values, not by real customer clusters.
- **Extreme outliers:** Num_Bank_Accounts, Num_Credit_Card, Interest_Rate, Num_Credit_Inquiries and Total_EMI_per_month contain values in the hundreds or thousands that flatten their histograms. They were not capped; decision trees split on thresholds, so they are fairly robust to this.
- **Correlations:** Monthly_Balance and Monthly_Inhand_Salary are positively correlated (about 0.6), while Delay_from_due_date is negatively correlated with both (about -0.2 to -0.3). Most other numeric pairs are close to zero, so multicollinearity is low.
- **Selected features:** the 15 features kept include salary, balance, payment delay, the engineered debt-to-income ratio, credit mix, minimum-payment behaviour, loan counts and payday loans. Payment discipline and credit mix stand out.

4. Model and Settings Tried

All runs use the same 15 selected features and the same train/test split, so differences come only from the model settings. Twelve settings of **DecisionTreeClassifier** were evaluated (random_state=42):

- **Baseline:** default tree (Gini criterion, no depth limit).
- **Split criterion:** Gini versus Entropy (both unlimited depth).
- **Tree depth:** max_depth from 2 to 10 (9 values), to control over-fitting.
- **Selection rule:** the setting with the highest macro F1 on the test set becomes the final model.



Figure 3. Train vs test accuracy by max_depth (left) and top-10 feature importances of the final tree (right).

The unrestricted tree reaches 100% training accuracy but only 68.1% test accuracy, a clear sign of over-fitting. Shallow trees (depth 2-4) under-fit. Test accuracy only reaches the unrestricted tree's 0.681 at depth 10, so going deeper adds complexity without real gain.

5. Results

Comparison of all settings (test set, macro-averaged metrics, sorted by macro F1):

Setting	Train Acc	Test Acc	Precision	Recall	Macro F1	Specificity	AUC
entropy (selected)	1.000	0.681	0.663	0.666	0.664	0.820	0.743
max_depth=10	0.824	0.681	0.654	0.668	0.660	0.826	0.770
gini	1.000	0.681	0.661	0.658	0.658	0.819	0.739
Baseline	1.000	0.681	0.661	0.658	0.658	0.819	0.739
max_depth=9	0.791	0.669	0.642	0.666	0.650	0.825	0.766
max_depth=6	0.713	0.655	0.639	0.672	0.645	0.824	0.793
max_depth=5	0.692	0.649	0.628	0.659	0.639	0.816	0.783
max_depth=8	0.769	0.655	0.630	0.651	0.637	0.816	0.768
max_depth=7	0.745	0.649	0.623	0.644	0.630	0.813	0.784
max_depth=4	0.674	0.630	0.608	0.621	0.613	0.796	0.762
max_depth=3	0.657	0.620	0.598	0.609	0.603	0.789	0.747
max_depth=2	0.633	0.599	0.587	0.619	0.594	0.793	0.718

Final model: Decision Tree with the Entropy criterion: accuracy **0.681**, macro precision **0.663**, macro recall **0.666**, macro F1 **0.664**, specificity **0.820**, macro AUC **0.743**. Several settings tie on accuracy (0.681), but entropy has the best macro F1. The best AUC (0.793) belongs to max_depth=6, which ranks customers well but classifies slightly less accurately.

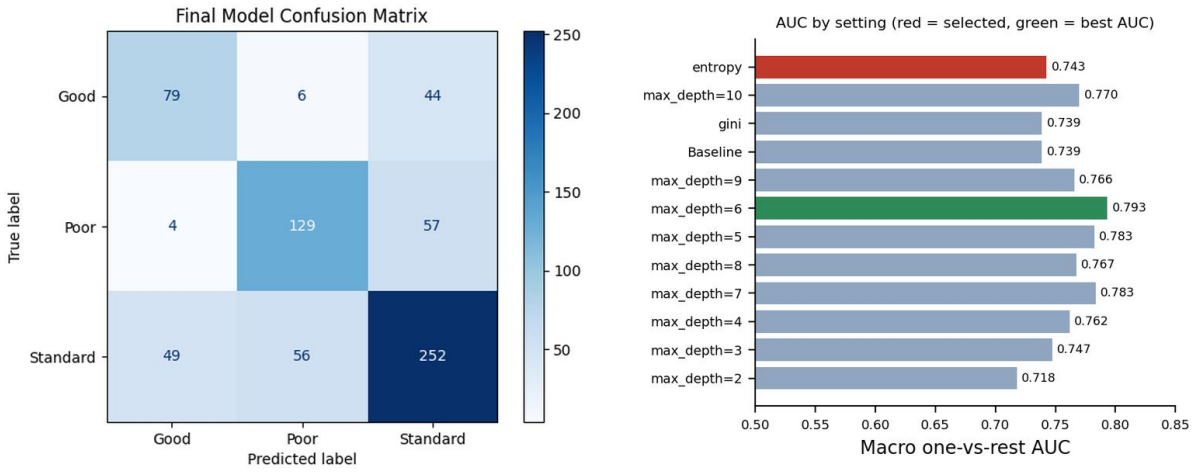


Figure 4. Confusion matrix of the final model on the 676 test records (left) and macro one-vs-rest AUC for every setting (right).

Class	Precision	Recall	F1	Test support
Good	0.598	0.612	0.605	129
Poor	0.675	0.679	0.677	190
Standard	0.714	0.706	0.710	357

Reading the confusion matrix: 460 of 676 test records are correct (79 Good, 129 Poor, 252 Standard). Almost all mistakes involve the middle class: Good-Standard (44 + 49) and Poor-Standard (57 + 56). Good and Poor are rarely confused with each other (6 + 4), so the model respects the natural ordering of the classes. Good is the hardest class (recall 0.61), partly because it has the fewest examples. The notebook reports AUC as a single macro one-vs-rest score (0.743 for the final model) rather than drawing per-class ROC curves, so Figure 4 shows AUC across all settings.

6. Conclusion and What I Learned

A Decision Tree using the **Entropy** criterion was the best of 12 settings, correctly classifying about **68% of customers** with macro F1 **0.664** and AUC **0.743**. Five features (Debt_to_Income_Ratio, Monthly_Balance, Delay_from_due_date, Monthly_Inhand_Salary and Credit_Mix_Good) account for roughly 88% of the tree's importance, so the model's decisions are driven by debt burden, cash position and payment discipline, which agrees with common sense.

What I learned

- Real data needs far more cleaning effort than modelling: placeholders, impossible values and missing data had to be fixed before anything worked.
- Accuracy can mislead on imbalanced data; macro F1, per-class recall and the confusion matrix give a fuller picture.
- An unpruned tree memorises the training data (100% vs 68%). Limiting depth reduces over-fitting and improves AUC, but the best metric depends on the goal.
- A simple engineered feature (debt-to-income ratio) became the single most important predictor.

Limitations and next steps

- Several numeric-looking columns (Annual_Income, Outstanding_Debt, Num_of_Loan, Num_of_Delayed_Payment, Changed_Credit_Limit, Amount_invested_monthly) stayed as text because the type check in the cleaning step did not match the new string dtype. They were one-hot encoded instead of used as numbers, which is why features such as Num_of_Loan_0 appear. Converting them properly and capping outliers should improve results.
- Each customer appears about 8 times, so a random row split can place the same customer in both train and test sets and give optimistic scores. A grouped split by Customer_ID would be fairer.
- Try Random Forest or Gradient Boosting, cross-validation, class weights and cost-complexity pruning, and add per-class ROC curves.